

Software Design Specification

Green Commute Planner

Prepared for	Software Engineering Course, Mahindra University
Prepared by	Group 22
Primary inputs	Statement of Work (SOW), SRS v1.0, SDS template
System type	Web/mobile application for sustainable commute planning
Architecture style	Layered service-based architecture
Date	8 April 2026

Team contributors referenced from project documents:

Pradham, Sohan - Project Management, requirements and documentation

Abhivarun, Shruthika - system design and architecture

Laasya Chowdary, Akshitha - application development

Pranavi, Shriyanka - testing, UI/UX and deployment

Document Information

Document title	Software Design Specification - Green Commute Planner
Document version	1.0
Prepared by	Group 22
Based on	SOW, SRS and SDS template
Status	Submission Ready

Version History

Version	Date	Revised by	Description
0.1	09 Feb 2026	Group 22	Statement of Work prepared and scope finalized
0.5	16 Mar 2026	Group 22	Software Requirements Specification completed
1.0	08 Apr 2026	Group 22	Software Design Specification completed from approved project scope and requirements

1 Introduction

1.1 Purpose

This Software Design Specification (SDS) describes how the Green Commute Planner system will be designed and implemented to satisfy the approved project scope and requirements. It translates the problem statement, use cases, functional requirements, and quality expectations into a concrete design covering architecture, modules, data, interfaces, exception handling, and operational quality.

1.2 Scope

The SDS applies to the Green Commute Planner web/mobile application. The design covers user authentication, route planning, route comparison, carbon-emission calculation, trip persistence, analytics, and administrator maintenance of transport emission factors. The document focuses on the software structure required for the academic prototype/MVP and assumes integration with an external mapping provider for route generation.

1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
API	Application Programming Interface
CO2	Carbon dioxide
Eco-score	Comparative sustainability score derived from commute mode metrics
GPS	Global Positioning System
MVP	Minimum Viable Product
Node.js	JavaScript runtime used for backend services
PostgreSQL	Relational database used for persistent storage
REST	Representational State Transfer
SDS	Software Design Specification
SOW	Statement of Work
SRS	Software Requirements Specification
UI/UX	User Interface / User Experience

1.4 References

- Statement of Work (Group 22, 09 Feb 2026)
- Software Requirements Specification for Green Commute Planner, Version 1.0
- Software Design Specification template provided for the assignment
- Google Maps Platform or Mapbox API documentation
- Node.js documentation
- PostgreSQL documentation
- IEEE-style software requirements/design documentation guidance

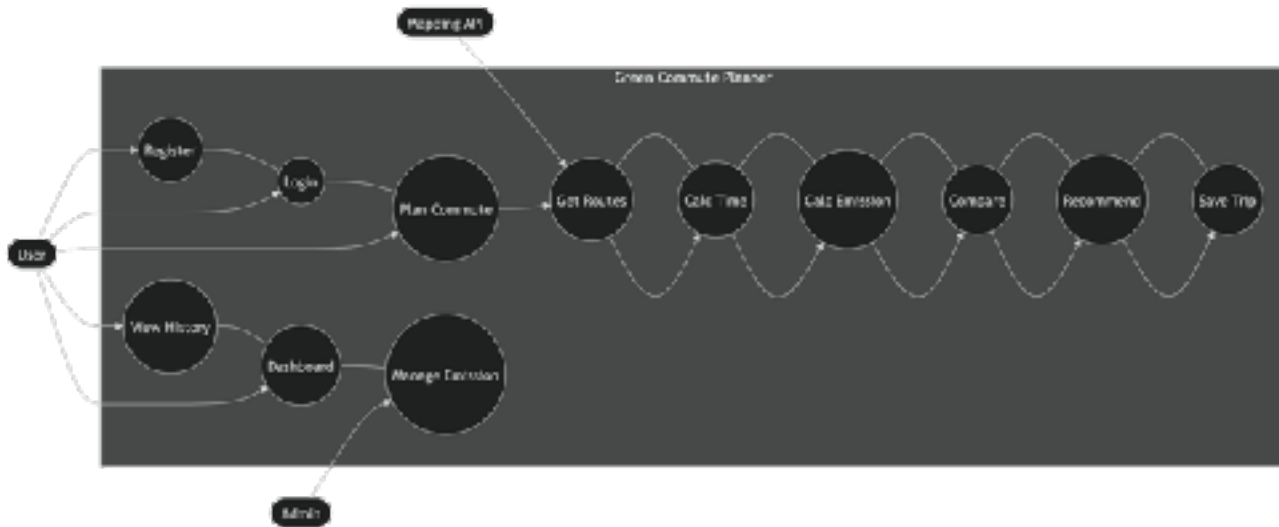
2 Use Case View

The design is driven by the central user journeys defined in the project requirements. The primary actors are User, Administrator, and External Mapping API. The most important use cases are registration/login, plan commute, view trip history, view carbon-savings dashboard, and manage emission factors.

2.1 Use Case Summary

Use case	Primary actor	Goal	Priority
UC-01 Register/Login	User	Create account and securely access personal features	High
UC-02 Plan Commute	User	Compare routes and choose the greenest option	High
UC-03 Save Trip	User	Persist selected route and commute details	High
UC-04 View Trip History	User	Review past trips and carbon savings	Medium
UC-05 View Dashboard Analytics	User	See trends and total emissions saved	Medium
UC-06 Manage Emission Factors	Administrator	Update transport mode factors used in calculations	Medium

2.2 Textual Use Case Diagram



2.3 Key Use Cases

UC-02 Plan Commute

Allows a logged-in user to enter source and destination details, retrieve route options, compare transport modes, and receive a recommended eco-friendly commute option.

1. User enters source and destination.
2. Client validates mandatory fields and sends request to backend.

3. Route Service requests route alternatives from Mapping API.
4. Emission Service computes estimated emissions for each transport mode.
5. Recommendation Engine ranks options using emissions, time, and distance.
6. System returns route cards showing time, distance, estimated emissions, and recommended option.
7. User may save a selected option as a trip record.

UC-04 View Trip History

Allows a logged-in user to view previously stored trips and aggregate carbon-savings information.

8. User opens dashboard/history screen.
9. Client requests user trips and summary analytics from backend.
10. Trip Service fetches trip records from PostgreSQL.
11. Analytics module computes cumulative savings and recent statistics.
12. System displays trip list, trend summary, and savings metrics.

UC-06 Manage Emission Factors

Allows an administrator to update emission factors used in route sustainability calculations.

13. Administrator opens admin panel.
14. System verifies admin role.
15. Administrator edits transport mode factor values.
16. Admin Service validates ranges and units.
17. System stores updated values and records audit metadata.
18. Updated factors are applied to future route comparisons.

3 Design Overview

The Green Commute Planner follows a layered service-based design. A presentation layer handles web/mobile interaction, an application layer coordinates use cases, a domain/service layer implements business rules, and a data/integration layer manages persistence and external API communication. This separation keeps the design modular, testable, and maintainable while fitting the academic project scope.

3.1 Design Goals and Constraints

- Support environmentally aware route comparison with a simple and intuitive user experience.
- Keep business logic modular by separating authentication, routing, emission calculation, trip management, analytics, and administration.
- Use Node.js for backend implementation and PostgreSQL for persistent storage.

- Integrate with Google Maps or Mapbox APIs for route and distance retrieval.
- Support both web and mobile frontends using a common REST API backend.
- Deliver an MVP within the semester timeline while preserving extensibility for future enhancements.

3.2 Design Assumptions

- Users have network connectivity when planning routes.
- External mapping APIs return valid route alternatives and travel-time estimates.
- Emission factors are maintained in the database and can be updated by administrators.
- GPS-based current-location input is optional and depends on device permissions.
- The prototype targets moderate academic-scale load rather than internet-scale deployment.

3.3 Significant Design Packages

Package / module	Responsibility	Key elements
Presentation	Render screens, gather input, show routes, history and analytics	Login UI, Planner UI, Route Cards, Dashboard UI, Admin Panel
API / Controllers	Expose REST endpoints and coordinate requests	AuthController, RouteController, TripController, AnalyticsController, AdminController
Domain Services	Implement core business logic	AuthService, RouteService, EmissionService, RecommendationService, TripService, AnalyticsService
Integration	Communicate with external systems	Mapping API Client, Password Hashing Adapter, Token Provider
Persistence	Store and retrieve application data	UserRepository, TripRepository, TransportModeRepository, AuditRepository

3.4 Dependent External Interfaces

External interface	Used by	Purpose
Google Maps / Mapbox Directions API	RouteService	Fetch route options, time estimates, distance and mode details
HTTPS network stack	All client-server communication	Secure transport for requests and responses
Device GPS / location service	Presentation layer	Optional current-location input

3.5 Implemented Application External Interfaces

Method	Endpoint	Owner module	Description
POST	/api/auth/register	AuthController	Register a new user
POST	/api/auth/login	AuthController	Authenticate user and return session/token
POST	/api/routes/plan	RouteController	Submit source/destination and return route comparison
POST	/api/trips	TripController	Save selected trip
GET	/api/trips/history	TripController	Fetch user trip history
GET	/api/analytics/summary	AnalyticsController	Return dashboard metrics and emission savings
GET/PUT	/api/admin/emission-factors	AdminController	Read or update mode emission factors

4 Logical View

The logical design organizes the system into collaborating modules. The client app calls backend controllers over HTTPS. Controllers invoke domain services, which apply validation and business rules, interact with repositories, and call external integrations when required.

4.1 Design Model

- Presentation Layer - Screens for Login/Register, Route Planner, Route Comparison Results, Trip History Dashboard, and Admin Settings.
- Controller Layer - Thin REST controllers that validate request format, invoke services, and map results to JSON responses.
- Service Layer - Core business logic including route retrieval, emission calculation, route recommendation, trip persistence, analytics aggregation, and role-based administration.
- Repository Layer - Database access for users, trips, transport modes, and audit records.
- Integration Layer - Adapters for mapping providers, password hashing, and session/token management.

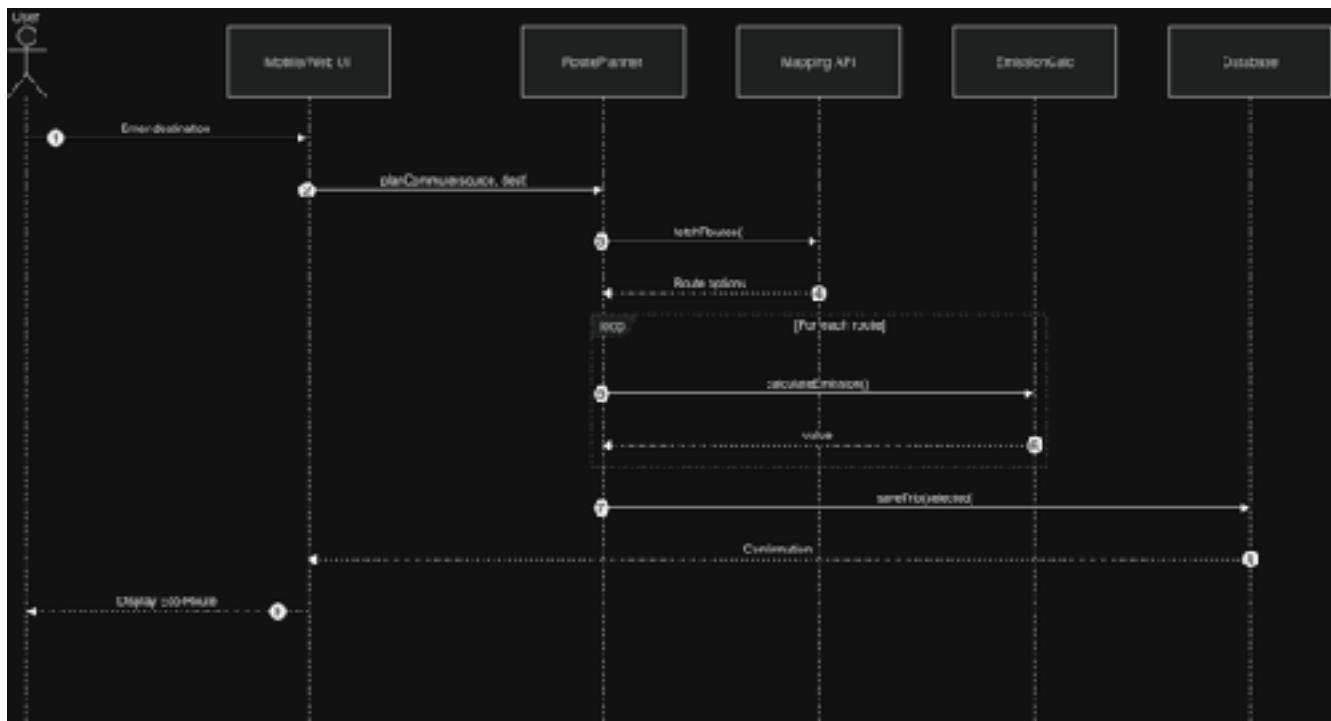
Recommended class/module relationships:



4.2 Use Case Realization

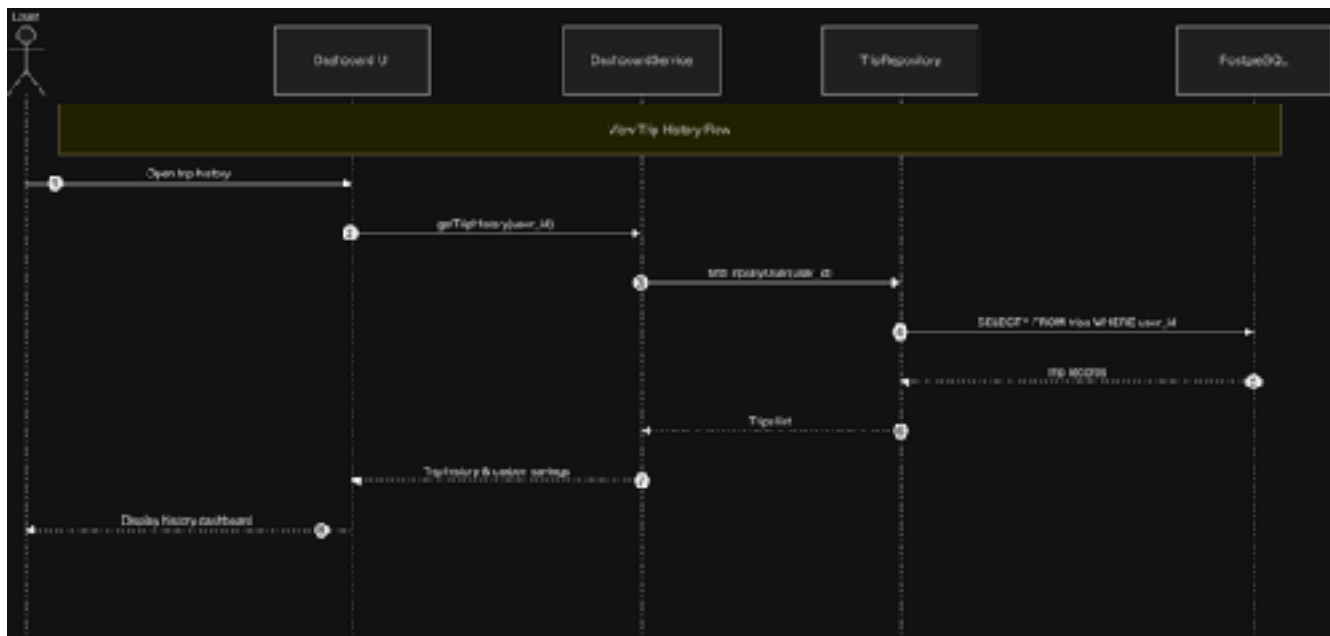
Realization of UC-02 Plan Commute

- 1. Planner UI collects source and destination.
- 2. RouteController validates request and forwards it to RouteService.
- 3. RouteService calls MappingApiClient to fetch candidate routes.
- 4. EmissionService calculates emissions per route using transport mode factors.
- 5. RecommendationService ranks routes using eco-score rules.
- 6. RouteController returns normalized route results to the client.
- 7. Planner UI highlights the recommended option and offers Save Trip.



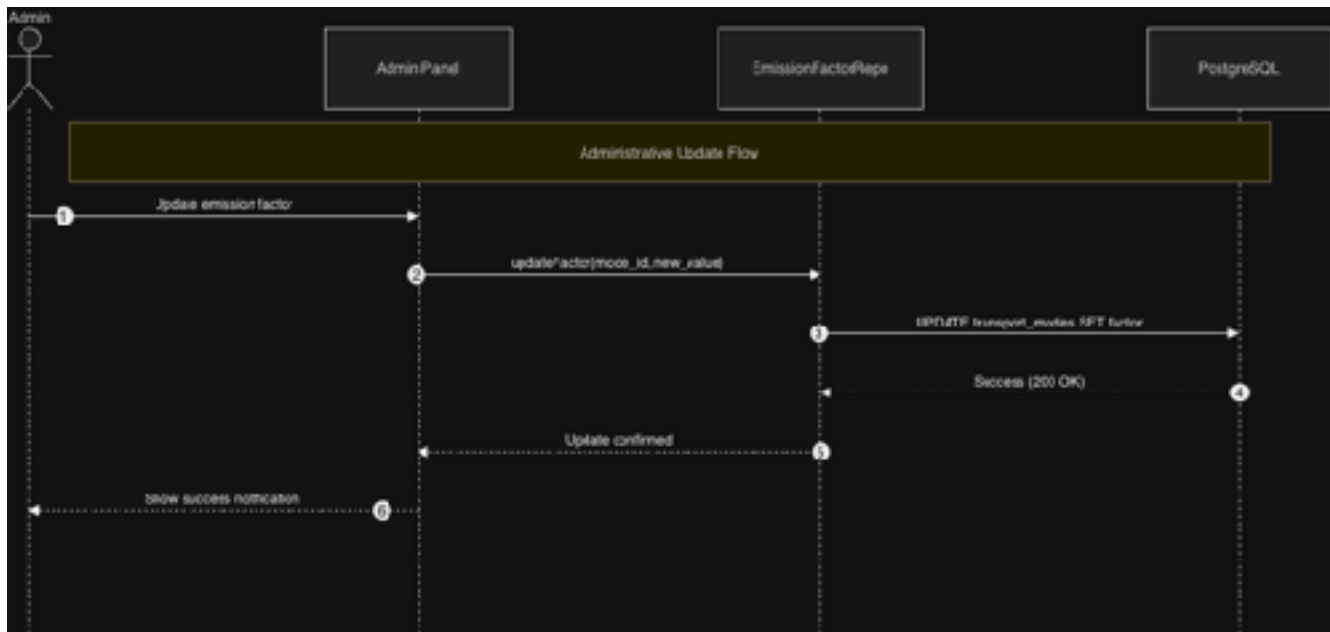
Realization of UC-04 View Trip History

- 1. Dashboard UI requests history for authenticated user.
- 2. TripController asks TripService for past trips.
- 3. TripService queries TripRepository for the user records.
- 4. AnalyticsService computes total trips, emissions, and cumulative savings.
- 5. Dashboard UI displays history and summary cards.



Realization of UC-06 Manage Emission Factors

- 1. Admin Panel sends factor updates to AdminController.
- 2. AdminController verifies administrator authorization.
- 3. AdminService validates value ranges and units.
- 4. TransportModeRepository updates the factor table.
- 5. AuditRepository stores who changed what and when.
- 6. The response confirms the updated factors for future calculations.



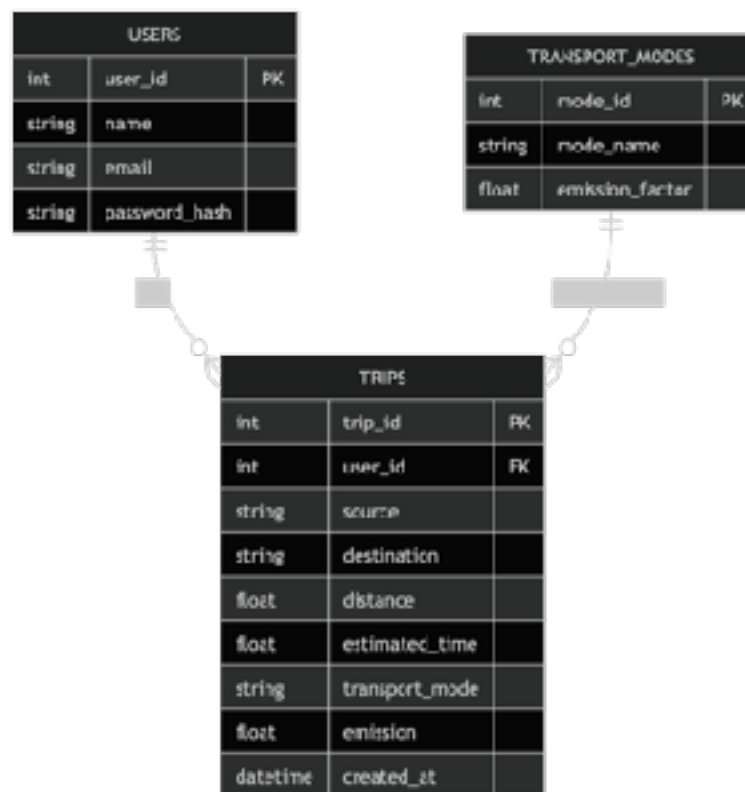
5 Data View

Persistent data is stored in PostgreSQL. The data model is intentionally small to match the MVP scope while supporting authentication, trip tracking, emissions logic, and administrator maintenance.

5.1 Domain Model

Core entities and relationships:

- User - owns many Trip records.
- Trip - belongs to one User and references one chosen TransportMode.
- TransportMode - stores emission factor and mode metadata used by EmissionService.
- EmissionFactorAudit - records administrator changes to transport mode factors.



5.2.1 Data Dictionary

Field	Type	Description	Constraints
user_id	UUID / serial	Unique user identifier	PK, not null
email	varchar	User login email	Unique, valid email, not null
password_hash	varchar	Hashed password value	Not null, never store plain text
role	varchar	User role: user/admin	Default user
trip_id	UUID / serial	Unique trip identifier	PK, not null
source	varchar	Trip start location	Not null
destination	varchar	Trip end location	Not null
distance_km	numeric	Travel distance in kilometers	Non-negative
duration_min	integer	Estimated travel time in minutes	Non-negative
estimated_emission	numeric	Calculated CO2 estimate for selected route	Non-negative
mode_name	varchar	Transport mode label	Unique
emission_factor	numeric	CO2 emitted per kilometer or equivalent unit	Non-negative

6 Exception Handling

The system handles validation, integration, authorization, and persistence failures through standardized error responses. Errors are logged on the backend with timestamps and request context while avoiding exposure of sensitive internals to end users.

Exception type	Thrown by	Typical cause	Handling strategy
ValidationError	Controllers / services	Missing or invalid source, destination, email, or factor value	Return 400 with user-friendly message
AuthenticationError	AuthService	Invalid credentials or expired session	Return 401 and redirect to login
AuthorizationError	AdminController / service	User attempts admin-only action	Return 403 and log event
ExternalApiError	RouteService / MappingApiClient	Mapping provider unavailable or timeout	Return graceful route-unavailable message and log retry context
DatabaseError	Repositories	Connection loss or query failure	Return generic failure message and log incident

7 Configurable Parameters

Parameter	Definition / usage	Dynamic?	Default
MAP_PROVIDER	Select Google Maps or Mapbox provider	No	Mapbox or Google Maps
MAP_API_KEY	Credential for route API calls	No	Environment-specific
JWT_SECRET / SESSION_SECRET	Authentication token signing secret	No	Environment-specific
DB_CONNECTION_STRING	Database connection configuration	No	Environment-specific
EMISSION_FACTOR_*	Per-mode emission factors used in calculations	Yes via admin panel	Stored in database
REQUEST_TIMEOUT_MS	Timeout for external API calls	No	3000-5000 ms

8 Quality of Service

8.1 Availability

The system targets high availability during normal academic use. The design minimizes downtime by keeping the application stateless at the service layer and storing persistent state in PostgreSQL. External mapping failures do not crash the application; instead, the user receives a controlled message.

8.2 Security and Authorization

- Passwords are stored as hashes, never as plain text.
- All client-server traffic uses HTTPS.
- Role-based authorization protects administrator-only functions.
- Input validation is applied at controller and service levels.
- Sensitive logs exclude raw passwords and secret keys.

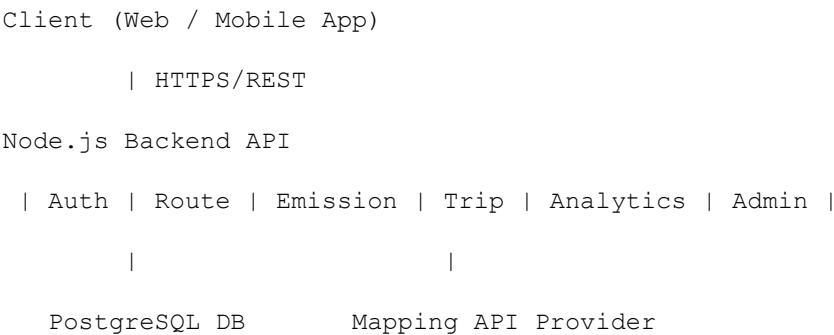
8.3 Load and Performance Implications

- Route-planning requests should return within about 3 seconds under normal conditions, subject to mapping API latency.
- Dashboard pages should load within about 2 seconds after login using indexed trip queries and pre-aggregated calculations where helpful.
- Trip-save operations should complete within about 1 second under normal load.
- Supporting up to 500 concurrent users requires efficient API handling, pooled database connections, and caching of static transport-mode factors.

8.4 Monitoring and Control

- Backend logs request failures, response times, authentication failures, and external API errors.
- Operational metrics include route-request latency, mapping API failure rate, trip-save success rate, and database connectivity health.
- Admin changes to emission factors are auditable through change logs.

Appendix A - Proposed Deployment View



Appendix B - Design Notes

This SDS is a design elaboration of the project scope and requirements already defined in the SOW and SRS. Where exact implementation details were not fixed in those documents, this specification adopts a reasonable layered Node.js + PostgreSQL design consistent with the stated technology constraints.